Instructor: Prof. Bartolomeo Stellato AI: Yanjun Liu

ORF522: Linear and Nonlinear Optimization

Fall 2025 Final Exam

December 15, 12:01am EST – December 18, 11:59pm EST

Exam Rules

1. Total time after download 24 hours.
2. Please write out and sign the following pledge at the bottom of this page: "I pledge my honor that I have not violated the Honor Code or the rules specified by the instructor during this exam."
3. You are allowed to use any material from the class (slides, code, books, etc.).
4. The exam is to be submitted electronically on Gradescope before 11:59pm. Please follow the `Gradescope guide` to take pictures and upload your answers. PDFs annotated with tablets are also accepted. Your PDF should contain the exported jupyter notebooks for your code. You cannot upload screenshots of your code.
5. The exam is not to be discussed with anyone except possibly the professor and the teaching assistants. You can only ask clarification questions as *public* (and preferable non-anonymous) on Ed forum. No emails.
6. The use of AI-based tools is forbidden. LLMs (ChatGPT, Claude, etc.), copilot chatbot and Google colab are not allowed on the final exam. You may use course materials, course references (including books), Python documentation and build-in autocompletes.
7. There are 5 problems worth a total of 100 points. There is no particular order of difficulty in the problems. Please be rigorous, brief and to the point in your answers. Good luck!

Name: _____

Honor code: _____

_____

_____

_____

1. *Fair resource allocation.* (15 points) Consider a resource allocation problem where a central planner must distribute resources among n agents subject to constraints such as budget limits, capacity, or fairness requirements. Suppose an initial proposal $x_0 \in \mathbf{R}^n$ is found to violate some of these constraints. To ensure fairness, the planner seeks a feasible allocation that minimizes the maximum adjustment to any single agent's share. This corresponds to projecting x_0 on a nonempty polyhedron $\mathcal{P} = \{x \in \mathbf{R}^n : Ax \leq b\}$ in the ℓ_∞-norm.

   (a) (3 points) Formulate this problem as an LP without norms.

   (b) (5 points) Derive the dual problem and simplify it as much as you can.

   (c) (7 points) Show that if $x_0 \notin \mathcal{P}$, then a hyperplane that separates x_0 from $\mathcal{P}$ can be constructed from the optimal solution of the dual problem. How would you interpret the coefficients of this hyperplane?

2. *Penalty methods for convex optimization.* (10 points) Consider a convex problem

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & g_i(x) \leq 0, \quad i = 1, \ldots, m. \end{array} \tag{1}$$

(a) (2 points) Express the dual function $q(\lambda)$ in terms of f and g_i for $i = 1, \ldots, m$.

The dual problem of (1) is

$$\begin{array}{ll} \text{maximize} & q(\lambda) \\ \text{subject to} & \lambda \geq 0. \end{array} \tag{2}$$

For the rest of this question, assume that strong duality holds for (1) and there is a unique dual optimal solution $\lambda^\star$.

(b) (4 points) For fixed $t > 0$, consider the unconstrained minimization problem

$$\text{minimize} \quad f(x) + t \max(0, g_1(x), \cdots, g_m(x)), \tag{3}$$

where x is the variable. Show that we can express (3) as

$$\begin{array}{ll} \text{minimize} & f(x) + ty \\ \text{subject to} & y \geq 0 \\ & y \geq g_i(x), \quad i = 1, \ldots, m. \end{array} \tag{4}$$

Then, find the Lagrange dual problem of (4) and express it in terms of the Lagrange dual function q for problem (1).

(c) (4 points) Use the result in part (b) to prove that if $t \geq \mathbf{1}^T \lambda^\star$, then the optimal values of (1) and (3) are the same.

3. *Contractive operators.* (20 points) Consider the following optimization problem

$$\text{minimize} \quad g(u) = f^*(A^T u) - b^T u,$$

where $u \in \mathbf{R}^m$. Suppose that f is CCP, continuously differentiable, and ∇f is γ-strongly monotone for $\gamma > 0$, and $I - \theta \nabla f$ is nonexpansive for a given $\theta > 0$.

(a) (2 points) Derive the dual problem.

(b) (4 points) Show that $\nabla g(u)$ is β-Lipschitz, specify the β in terms of the spectral norm of A, which is its largest singular value, defined as $\sigma_{\max}(A)$.

(c) (5 points) Show that, for a given $\alpha > 0$, the fixed point iteration of $(2I - \alpha \nabla f)^{-1}(I - \alpha \nabla g)$ is

$$\begin{aligned} x^{k+1} &= \underset{x}{\operatorname{argmin}}\ f(x) - (z^k)^T(Ax - b) \\ z^{k+1} &= \underset{z}{\operatorname{argmin}}\ -\alpha f(z) + (1/4)\|2z - z^k - \alpha A x^{k+1} + \alpha b\|_2^2. \end{aligned}$$

(d) (4 points) Assume that $0 < \alpha < 2/\beta$. Show that $(I - \alpha \nabla g)$ is contractive with factor of $\max\{|1 - \alpha\rho|, |1 - \alpha\kappa|\}$, define ρ and κ.

Hint: Use the fundamental theorem of calculus.

(e) (3 points) Show that the operator $(2I - \alpha \nabla f)^{-1}$ is contractive and derive its contraction factor as a function of α and θ.

(f) (2 points) Compute the overall contraction factor of the composed operator $(2I - \alpha \nabla f)^{-1}(I - \alpha \nabla g)$.

4. *Multi-agent coordination.* (25 points) A system consists of n cooperative agents. Each agent i chooses a local decision $x_i \in \mathbf{R}^{p_i}$ and the group jointly uses a shared coordination decision $y \in \mathbf{R}^q$. Each agent has its own private cost function $f_i(x_i, y)$. The group wants to solve the multi-agent cooperative problem

$$\text{minimize} \quad \frac{1}{n}\sum_{i=1}^{n} f_i(x_i, y) + r(y), \tag{5}$$

where $r(y)$ represents the cost of the coordinated decision. Each f_i is of the form

$$f_i(x_i, y) = \frac{1}{2}x_i^T Q_i x_i + \frac{1}{2\mu}\|y\|^2 + x_i^T(y + c_i),$$

where both $Q_i \in \mathbf{S}_{++}^{p_i \times p_i}$ and $c_i \in \mathbf{R}^{p_i}$ are known, each Q_i satisfies $\mu I \preceq Q_i \preceq LI$ for some known μ and L, and $p_i = q$ for all i. Consider the 1-norm function $r(y) = \|y\|_1$. Use the following code to generate the problem data:

```
def random_pd_matrix(n, lambda_min, lambda_max, seed=42):
    np.random.seed(seed)
    M = np.random.randn(n, n)
    Q, _ = np.linalg.qr(M)
    eigvals = np.random.uniform(lambda_min, lambda_max, n)
    return Q @ np.diag(eigvals) @ Q.T

np.random.seed(42)
mu, L, dim, n = 1, 5, 1000, 20
Qs = [random_pd_matrix(dim, mu, L) for _ in range(n)]
cs = [np.random.randn(dim) for _ in range(n)]
y_init = np.random.randn(dim)
```

(a) (3 points) Define $\phi_i(y) = \inf_{x_i \in \mathbf{R}^{p_i}} f_i(x_i, y)$ and consensus set

$$\mathcal{C} = \{(y_1, \cdots, y_n) \in \mathbf{R}^{q \times n} \mid y_1 = \cdots = y_n\}.$$

Write (5) as a consensus problem as a function of ϕ and $\mathcal{C}$.

(b) (6 points, Coding) Compute $\phi_i(y)$ in closed form and use it to formulate (5) as a convex optimization problem in y only. Solve the problem using CVXPY.

(c) (5 points) Let $h(y_1, \cdots, y_n) = r(y_1) + \mathcal{I}_{\mathcal{C}}(y_1, \cdots, y_n)$. For any $\alpha > 0$, compute $\mathbf{prox}_{\alpha h}(y_1, \cdots, y_n)$. You can express your result using $\mathbf{prox}_{\beta r}$ for some $\beta > 0$.

(d) (5 points) Write down the iterations of a distributed proximal gradient method with parameter $\alpha > 0$ applied to your formulation in (a). Your final answer should involve only explicit iterations (i.e., no implicit **prox** or argmin operators).

(e) (6 points, Coding) Implement your algorithm in part (d) with step size $\alpha = 2/L$. Produce two plots (both in log scale) comparing against the CVXPY solution from part (b):

- Suboptimality $|f(y^k) - f^\star|$ vs. iteration number.
- Distance to optimum $\|y^k - y^\star\|_2$ vs. iteration number.

Then, implement a parallel version of your algorithm and compare the execution times. Are you able to get speedups? Why or why not?

Hint: Use `ThreadPoolExecutor` from `concurrent.futures` to parallelize the gradient computations across agents. Example usage:

```
from concurrent.futures import ThreadPoolExecutor

def compute_gradient(i):
    # compute gradient for agent i
    return gradient_i

with ThreadPoolExecutor(max_workers=num_threads) as executor:
    gradients = list(executor.map(compute_gradient, range(n)))
```

5. *GPU datacenter cooling.* (30 points) Modern GPU datacenters require active cooling to ensure safe operations. Consider the problem of cooling a large-scale datacenter. Let $x_t \in \mathbf{R}^n$ denote list of temperatures recorded in key areas of the datacenter at time t, and let the scalar $u_t \in \mathbf{R}$ denote the cooling actuation (e.g., chilled-water flow, fan speeds). While the true temperature dynamics are highly nonlinear, we consider a linearized model of the type:

$$x_{t+1} = Ax_t + b_t u_t,$$

for given $A \in \mathbf{R}^{n\times n}$ and $b_t \in \mathbf{R}^n$. Because of model simplifications and fluctuating thermal loads and coolant quality, the cooling-efficiency described by vector b_t is uncertain. To be specific,

$$b_t \in \mathcal{U}_{b_t} = \left\{ b_t = \bar{b} + \delta_t, \quad \|\delta_t\|_2 \le \rho \right\},$$

where $\bar{b}$ is the nominal value, the uncertain parameters at time t are $\delta_t \in \mathbf{R}^n$, and the uncertainty set radius is $\rho > 0$. Let

$$\mathcal{X} = \{x \mid Cx \le d\},$$

denote a safe-temperature set that avoids dangerous overheating for some known $C \in \mathbf{R}^{q\times n}$ and $d \in \mathbf{R}^q$. Given a time horizon T, the goal is to compute the most efficient sequence of cooling actions $u_0, \ldots, u_T$, that minimize the total cooling energy used such that the temperature of the system stays safe,

$$\begin{array}{ll} \text{minimize} & \sum_{t=0}^{T-1} u_t^2 \\ \text{subject to} & x_{t+1} = Ax_t + b_t u_t, \quad t = 0, \ldots, T-1 \\ & x_t \in \mathcal{X}, \quad \forall b_t \in \mathcal{U}_{b_t}, \quad t = 0, \ldots, T, \\ & x_0 = x^{\text{init}}, \end{array}$$

where x^{init} is the initial state, i.e., the initial temperatures. Use the following code to generate the problem data:

```
import numpy as np
np.random.seed(42)  # For reproducibility

# System parameters
n, T, rho = 2, 5, 0.1
sigma = 0.40  # For Gaussian uncertainty samples

A = np.array([[1.1, 0.2],
              [0.0, 0.9]])
b_bar = np.array([1.0, 0.5])
x_init = np.array([10.0, 5.0])

# Safe temperature constraints: Cx <= d
```

```
C = np.array([[1.0, 1.0],
              [0.5, -0.5]])
d = np.array([25.0, 8.0])
```

(a) (2 points) Write the nominal dynamics, where $b_t = \bar{b}$ as $X = \mathcal{A}x^{\text{init}} + \mathcal{B}U$ where the sequence of states is $X = (x_1, \ldots, x_T) \in \mathbf{R}^{nT}$ and the sequence of inputs is $U = (u_0, \ldots, u_{T-1}) \in \mathbf{R}^T$. Define what matrices $\mathcal{A}$ and $\mathcal{B}$ are.

(b) (3 points) For each t, write the uncertain constraint as a function of x_0 and $U = (u_0, \ldots, u_{T-1}) \in \mathbf{R}^T$, $\mathcal{A}$ and $\mathcal{B}$. Which types of inequalities do you obtain?

Hint: Define $x_t = E_t X$.

(c) (3 points) For each time, and each constraint of $\mathcal{X}$ $c_l^T x_t \leq d_l$ for $l = 1, \ldots, q$, write the robust counterpart by removing the explicit dependency on δ_t.

Hint: Define $\Delta_t = (\delta_0, \ldots, \delta_t)$ and $\mathcal{U}_{\Delta_t} = \{(\delta_0, \ldots, \delta_t) \mid \|\delta_i\|_2 \leq \rho,\ i = 0, \ldots, t\}$.

(d) (4 points, Coding) Write the complete robust optimization problem to solve the datacenter cooling problem. Implement it in CVXPY using $\rho = 0.1$ and report the optimal objective value and control sequence $U^\star$.

The robust formulation provides worst-case guarantees over the uncertainty set $\mathcal{U}_{b_t}$. In practice, we often want probabilistic guarantees instead. A *chance-constrained* formulation requires

$$\mathbf{P}(x_t \in \mathcal{X}) \geq 1 - \epsilon, \quad t = 0, \ldots, T,$$

for some target violation probability $\epsilon \in (0, 1)$. The parameter ρ in the robust formulation controls conservativeness: larger ρ leads to more robust (but potentially more costly) solutions. The following questions explore the relationship between ρ and the empirical violation probability.

(e) (3 points, Coding) Assume the true uncertainty follows a Gaussian distribution $\delta_t \sim \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 0.40$. Generate a validation dataset of $N_{\text{val}} = 500$ samples. Compute the empirical maximum violation probability for the robust solution $U^\star$ from part (d), defined as

$$\max_{t=0,\ldots,T} \mathbf{P}(x_t \notin \mathcal{X}) = \max_{t=0,\ldots,T} \mathbf{P}\left(\max_{l=1,\ldots,q} c_l^T x_t - d_l > 0\right).$$

(f) (4 points, Coding) The robust optimization parameter ρ controls the conservativeness of the solution. Tune ρ using the validation dataset to find the value that achieves the largest maximum violation probability that is still below a target $\epsilon = 0.10$. For each candidate ρ, solve the robust problem, then compute the maximum violation probability across all constraints and all time steps. Use the grid $\rho \in \{0.10, 0.15, 0.20, 0.25, 0.30, 0.35\}$ and report the selected $\rho^\star$ and the corresponding objective value. Finally, generate a separate test dataset of

$N_{\text{test}} = 10000$ samples and report the maximum violation probability of the selected solution on this test set.

(g) (3 points) An alternative to robust optimization is the *scenario approach*, which directly samples uncertainty realizations. Given N sample trajectories $(\hat{\delta}_0^k, \ldots, \hat{\delta}_{T-1}^k)$ for $k = 1, \ldots, N$, formulate the scenario optimization problem that enforces the constraints for all sampled scenarios.

Hint: Define $\hat{\Delta}_t^k = (\hat{\delta}_0^k, \ldots, \hat{\delta}_t^k)$.

(h) (2 points) Using the scenario approach theory, compute the number of samples N required to guarantee that the solution satisfies $\mathbf{P}(x_t \in \mathcal{X}) \geq 1 - \epsilon$ for all t with confidence $1 - \beta$, where $\epsilon = 0.10$ and $\beta = 10^{-6}$.

(i) (3 points, Coding) Solve the scenario optimization problem using the computed N. Report the optimal objective value, the optimal control sequence $U^\star$, and compare to the results obtained in part (f).

(j) (3 points, Coding) The theoretical bound on N is often conservative. Tune N using the validation dataset to find the value that achieves the largest maximum violation probability that is still below the target $\epsilon = 0.10$. Use the grid $N \in \{10, 20, 40, 60, 100, 150\}$. Report the selected $N^\star$, the corresponding objective value, and the test set violation probability. Compare the result to the tuned robust solution from part (f).